SessionBound: Database-Enforced Task-Bound Sessions for Enterprise AI Agents

Minmin Wu
China Telecom Global Limited
wuminmin@chinatelecomglobal.com

***Abstract*—Enterprise AI agents are increasingly useful for internal analysis, audit, compliance review, and operational investigation. They are good at writing SQL on the fly, but enterprises cannot trust them with raw database access. These tasks are typically approved above the database layer by business users, managers, data owners, or compliance teams, but executed below that layer through open-ended SQL generated by an agent. This creates a missing systems layer: enterprise task approval must be translated into a database execution context that is bounded, budgeted, and auditable. SessionBound bridges this trust gap by turning a business approval into a short-lived, budgeted, and auditable database session.**

We present SessionBound, a control-plane and database-runtime architecture that turns approved enterprise tasks into budgeted database sessions for AI agents. A SessionBound control plane defines task templates, accepts task applications, applies grants and approvals, assigns TTLs and budgets, and issues signed task tokens. A database runtime, SessionBoundDB, binds an approved task token to a short-lived agent session and enforces safe views, row scope, denied fields, operation limits, structural SQL policy, query budgets, disclosure budgets, and query receipts at SQL execution time. The hardening prototype includes both API-layer AST validation and an experimental PostgreSQL hook path for database-resident structural SQL enforcement.

The current evaluation validates 24 of 24 canonical scenarios and a 28-case adversarial SQL suite with 22 blocked cases, 5 allowed safe-view analytical cases, and 1 known limitation. Performance results separate the security model from the reference runtime. On the default synthetic seed, safe-view-only p50 latency is 13.3–18.1 ms and full SessionBound p50 latency is 14.6–18.6 ms, indicating that small-scale overhead is dominated by the safe-view/session runtime rather than by receipt-chain insertion or budget updates. A 100k-row synthetic scale sweep exposes multi-second latency in the current PL/pgSQL wrapper path, motivating a parser-, planner-, or executor-hook implementation for production deployments.

SessionBound does not require the database to parse natural-language task descriptions, nor does it trust the agent or an LLM to correctly interpret or obey the task. The agent remains free to generate SQL, join safe business objects, aggregate, rank, and drill down. The database decides whether each attempt stays inside the approved boundary. This design targets a setting distinct from authenticated delegation, task-scoped service-operation authorization, and data-product marketplace access: enterprise-internal analytical work where fixed SaaS screens are too rigid, direct database access is too broad, and approved tasks should become temporary, budgeted, receipt-bearing database sessions.

Code availability. The prototype source code, synthetic evaluation dataset, validation reports, benchmark outputs, and LaTeX source for the paper are available at https://github.com/SessionBound/sessionbound.

Preprint availability. The arXiv version is available at https://arxiv.org/abs/2607.00751.

***Index Terms*—AI agents, access control, database security, dependable computing, secure computing, authorization, SQL security, data governance, auditability.**

I. Introduction

AI agents change how enterprise users interact with data. A user no longer needs to know which report exists, which dashboard to open, or which API endpoint to call. Instead, the user can ask an agent:

> Analyze June 2026 travel reimbursements for the Sales department. Find unusual merchants, repeated claims, employee-level monthly totals, and claims near approval thresholds. Exclude salary and bank data.

This is exactly the kind of task agents are good at: temporary, exploratory, cross-table, and under-specified. It is also exactly the kind of task traditional enterprise software handles poorly. Building a permanent SaaS page or API for every low-frequency internal analysis task is expensive. Giving the agent raw database access is unsafe. Relying on prompts is not a security boundary. Relying on the application layer alone becomes fragile when SQL is generated dynamically.

The industry and research communities have already recognized several adjacent problems. OAuth 2.0 provides the standard delegated authorization model for web APIs [1]. Agent identity systems make agents visible and governable as non-human identities. Authenticated delegation frameworks let users verifiably delegate authority to agents [2]. Task-scoped authorization systems such as PAuth ask whether a specific API or tool operation is implied by the user's task [3]. Data-product governance systems such as Data Product MCP connect enterprise data marketplaces with MCP so that agents can discover, request access to, and query governed data products [4], [5]. Database security

systems such as Oracle Deep Data Security show that fine-grained policies should be enforced close to the data [6], [7]. Large-scale authorization systems such as Zanzibar show how relationship-based permission checks can be made consistent and global [8].

These are necessary pieces, but they leave a gap for enterprise-internal analytical agents. In real organizations, a manager or data owner does not log into a database to write row-level policies for every agent task. A business user applies for or delegates a task. A manager, data owner, or compliance workflow approves it. A data platform has already registered safe business views. A system must convert the approved task into a structured execution boundary that the database can enforce, while still allowing the agent to freely generate useful analytical SQL.

This paper argues for the following principle: business users approve tasks, not database policies. Agents generate SQL, but databases enforce the approved boundary. This paper frames the problem as a database-enforced authorization boundary for enterprise AI agents, rather than as an agent-planning or multi-agent coordination problem.

SessionBound separates three concerns.

1. Task control plane. Defines task templates, accepts task applications, evaluates grants and approvals, assigns budgets and TTLs, and signs task tokens.
2. Agent execution. Lets an LLM-powered agent decide what SQL to try, which hypotheses to test, and how to construct a temporary workspace.
3. Database runtime. Binds the signed task token to a database session and enforces safe views, scope, denied fields, operation limits, query budgets, disclosure budgets, and receipts.

The central design rule is that agents may explore only within a structured boundary that the database enforces deterministically.

This makes SessionBound a contract layer between enterprise approval and agentic database execution. The control plane records what the organization approved; the runtime enforces how that approval may be spent through SQL. This contract is not a database policy written by a DBA, nor a natural-language instruction interpreted by an LLM. It is a structured, signed, and accountable representation of an approved business task that both the agent runtime and the database can interpret deterministically.

SessionBoundDB is our PostgreSQL-based reference runtime for SessionBound. It does not ask an LLM whether a query is safe. It does not depend on the agent correctly interpreting the task. It does not trust mutable session variables set by the agent. Instead, it validates a signed task token, exposes only registered safe views, rejects raw table access and denied fields, tracks query and disclosure budgets, and records receipts for allowed and denied attempts.

For compatibility with the existing demo implementation, the PostgreSQL prototype currently keeps the SQL schema name `taskbound`.

A. Contributions

This paper makes five contributions.

1. Enterprise task approval as the missing layer. We identify a gap between agent delegation and database enforcement: real enterprises approve tasks, scopes, and budgets above the database, but need those approvals translated into database-enforceable execution boundaries.
2. SessionBound control plane. We propose a task control plane with task templates, task applications, task approvals, grants, TTLs, budgets, and signed task tokens. This control plane lets business and data-governance workflows define what work is authorized without requiring managers to write database policies.
3. Budgeted database sessions. We define a database session model in which an approved task token binds an agent to safe views, row scope, denied fields, allowed operations, TTL, query budget, disclosure budget, and audit receipt policy.
4. LLM-independent enforcement for open-ended SQL. SessionBoundDB allows agents to generate SQL freely inside a boundary, but the database enforces that boundary deterministically. The database does not parse the natural-language task and does not rely on an LLM to judge safety at query time.
5. PostgreSQL prototype and hardening analysis. We implement SessionBoundDB as a PostgreSQL-based runtime for SessionBound and demonstrate it on enterprise travel reimbursement analysis. The hardening prototype includes a `sessionbound_guard` C extension loaded through `shared_preload_libraries` that uses PostgreSQL's `post_parse_analyze_hook` to reject unsafe SQL structures before dynamic execution. We also analyze anti-evasion scope, budget semantics, schema drift, credential-token binding, receipt structure, and prototype-to-production hardening requirements.

II. Motivation: Enterprise Tasks Are Approved Above the Database

A. SaaS is strong at fixed workflows and weak at temporary analysis

Traditional SaaS applications are excellent for stable workflows. Expense systems have pages for submitting claims, approving them, and exporting reports. CRM systems have pages for accounts, opportunities, and pipeline forecasts. HR systems have forms for updating employee details. These interfaces are valuable because they encode product experience, business rules, and human responsibility.

But enterprises also contain many tasks that are temporary, analytical, and low-frequency:

- audit unusual reimbursement patterns for one month;
- compare vendor concentration across departments;
- investigate repeated support escalations for a customer segment;
- find transactions near policy thresholds;
- review access logs for a compliance incident;
- build a temporary workspace for a manager's operational question.

These tasks are too specific to justify permanent SaaS pages, but too sensitive to give an agent raw database access. They often require joins, aggregation, ranking, drill-down, and follow-up queries. The exact SQL cannot be known in advance.

SessionBound targets this setting.

B. Enterprise approval is not database policy authoring

In a realistic enterprise, a manager or data owner should not be expected to write SQL policies. A business user might request:

> **Task type:** monthly expense anomaly review
> **Scope:** June 2026, Sales department
> **Purpose:** travel reimbursement audit
> **Sensitivity:** exclude salary, phone, and bank account fields
> **Budget:** 20 SQL attempts, 5,000 result rows, bounded entity exposure
> **TTL:** 30 minutes

A manager or data owner can decide whether this task is appropriate. A data platform can provide approved safe views. A compliance system can require tighter budgets. But none of these people should need to edit database roles, row-level security predicates, or cell-level policies for every agent session.

SessionBound therefore separates task governance from database enforcement:

```
Business users approve tasks.
Data platforms register safe views.
Control planes sign task tokens.
Databases enforce boundaries.
Agents explore freely inside
those boundaries.
```

C. Running example: travel reimbursement anomaly review

We use travel reimbursement review as a running example. A finance analyst or department manager wants an agent to analyze June travel reimbursements for a department. The agent may need to:

- rank high-value claims;
- group expenses by merchant, city, category, and employee;
- compute monthly employee totals;
- find claims close to approval thresholds;
- drill down into suspicious patterns;
- propose follow-up review actions.

The agent should not see salary, bank account, phone number, or raw internal tables. It should not query another department, another month, or another tenant. It should not run DDL or arbitrary updates. It should not paginate through all employees and reconstruct an unrestricted dataset. Its access should expire quickly and leave an audit trail.

A fixed endpoint such as `get_expense_summary`, parameterized by department and month, is too narrow. A broad database account is too dangerous. A prompt instruction such as "do not access salary" is not a security mechanism. SessionBound's answer is to approve a task and bind it to a database session with safe views, scope, budgets, and receipts.

III. Threat Model

SessionBound assumes the upper-layer agent is useful but not trusted. The agent may be confused, overconfident, prompt-injected, malicious, or simply wrong. The database boundary must remain meaningful even when the agent tries the wrong thing.

A. Trusted components

For this prototype, we trust:

- the task control plane that defines templates, evaluates grants, records approvals, and signs task tokens;
- the credential broker that issues short-lived database credentials;
- the database runtime that validates tokens and enforces policies;
- the cryptographic signing key or signing service used for task tokens;
- the integrity of registered safe views and runtime functions.

In production, the signing path should be backed by a KMS, JWKS, hardware-backed key service, or enterprise identity infrastructure. The prototype uses a simpler signing path for demonstration.

B. Untrusted or partially trusted components

We do not trust:

- the agent's prompt;
- the agent's reasoning;
- the agent's SQL generation;
- the agent's natural-language interpretation of the task;
- user-provided or database-provided text that may contain prompt injection;
- ordinary client-controlled session variables;
- application-layer filtering as the final boundary.

This is the main difference from designs that require an LLM to infer the precise intended operation from natural language. SessionBound does not ask the database to understand the user's natural-language intent. The database consumes structured claims from a signed token.

C. In-scope threats

SessionBoundDB is designed to mitigate:

- Sensitive field access: attempts to read salary, bank account, phone, identity number, credentials, or private notes.
- Raw table access: attempts to bypass safe views and query application schemas directly.
- Scope escape: attempts to read other tenants, months, departments, projects, or users.
- Unauthorized mutations: write, DDL, or other high-impact operations outside controlled commands.
- Prompt injection effects: data or user text instructs the agent to ignore policy or reveal secrets.
- Budget evasion: repeated queries, pagination, or alternate SQL forms attempt to disclose more detail rows than the task allows.
- Credential-token mismatch: attempts to combine one runtime credential with another task token.

D. Out of scope

The prototype does not solve:

- malicious database administrators;
- compromised database host operating systems;
- compromised signing infrastructure;
- side-channel attacks;
- complete formal verification of SQL equivalence;
- production-grade planner/executor-level SQL enforcement;
- cross-database federation;
- full differential privacy for statistical releases;
- arbitrary semantic inference across all allowed query answers.

SessionBound reduces and accounts for what an agent can discover. It does not claim to eliminate all inference.

IV. SessionBound Model

SessionBound has two major layers: a task control plane and a database runtime. Figure 1 shows the main SessionBound control-plane and runtime boundary.

A. Task template

A task template is defined by a data platform or application team. It describes a class of work that can be safely delegated to agents once scoped and approved.

Example:

```
{
  "task_type": "monthly_expense_anomaly_review",
  "purpose": "travel_reimbursement_audit",
  "allowed_views": ["expenses", "employees", "departments"],
  "denied_columns": [
    "employees.salary",
    "employees.bank_account",
    "employees.phone"
  ],
  "required_scope": ["tenant_id", "expense_month", "department_id"],
  "allowed_operations": ["SELECT"],
  "default_ttl_minutes": 30,
  "default_budget": {
    "max_queries": 20,
    "max_result_rows": 5000,
    "max_unique_entities": {
      "expense_id": 5000,
      "employee_id": 200
    }
  }
}
```

The template is not generated by the agent. It is part of enterprise governance.

B. Task application

A task application is a concrete request to perform work under a template.

```
{
  "task_type": "monthly_expense_anomaly_review",
  "requested_by": "user:alice",
  "actor": "agent:travel-expense-analyst",
  "scope": {
    "tenant_id": "company_a",
    "expense_month": "2026-06",
    "department_id": "dep_sales"
  },
  "reason": "Review unusual travel reimbursement patterns."
}
```

C. Task approval

A task approval evaluates whether the requester may run this task with this scope and budget. Approval may be automatic, role-based, or manual. For sensitive tasks, a manager, data owner, or compliance reviewer may approve it.

The approval does not require the approver to write SQL policies. It decides whether this scoped task should exist.

D. Task token

A signed task token is the structured authorization object consumed by the database runtime.

```
{
  "task_id": "task_456",
  "task_type": "monthly_expense_anomaly_review",
  "delegator": "user:alice",
  "actor": "agent:travel-expense-analyst",
  "credential_id": "cred_123",
  "tenant_id": "company_a",
  "purpose": "travel_reimbursement_audit",
  "allowed_views": ["expenses", "employees", "departments"],
  "denied_columns": [
    "employees.salary",
    "employees.bank_account",
    "employees.phone"
  ],
  "row_scope": {
    "expense_month": "2026-06",
    "department_id": "dep_sales"
  },
  "operations": ["SELECT"],
  "budget": {
    "max_queries": 20,
    "max_result_rows": 5000,
    "max_unique_entities": {
      "expense_id": 5000,
      "employee_id": 200
    }
  },
  "safe_view_registry_version": "travel-demo-v1",
  "policy_version": "expense-review-v1",
  "audience": "sessionbounddb",
  "expires_at": "2026-06-24T10:30:00Z",
  "nonce": "..."
}
```

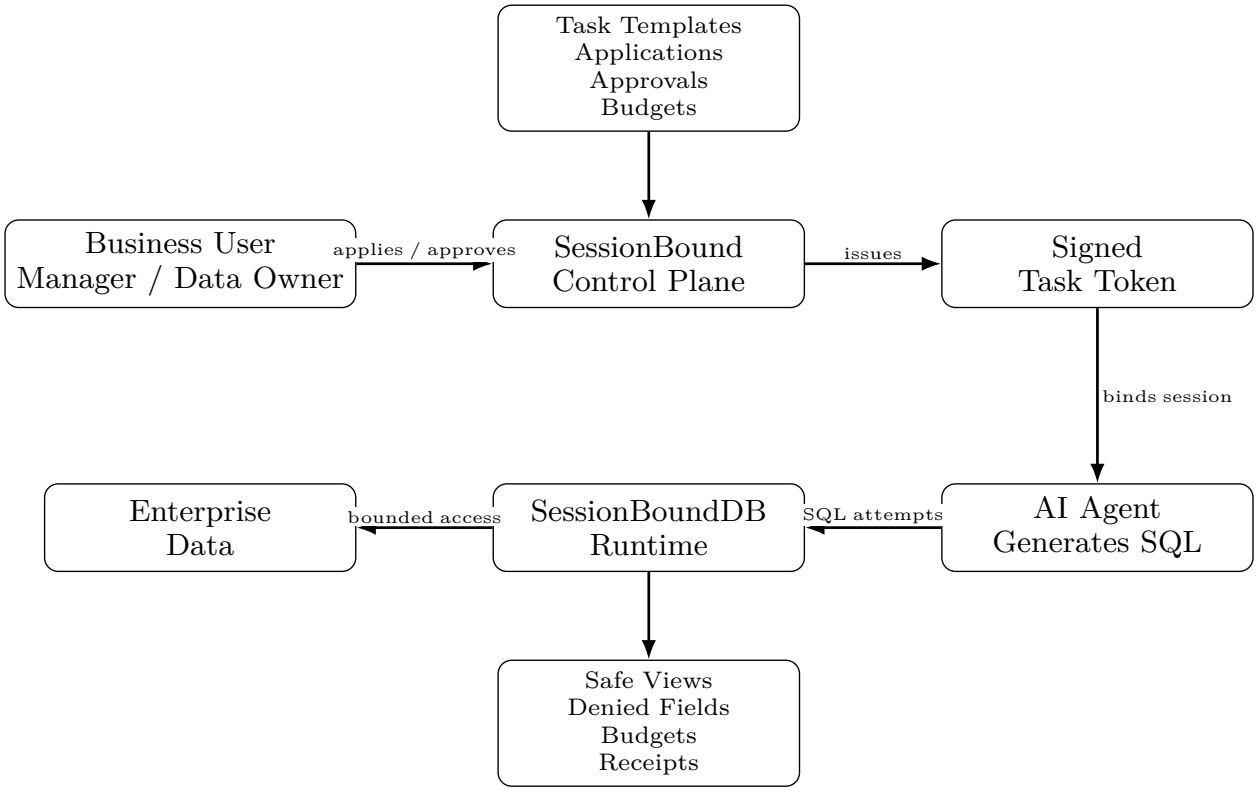

Fig. 1. SessionBound architecture. The control plane turns enterprise task approval into a signed task token; the agent may generate SQL freely, but SessionBoundDB enforces safe views, denied fields, budgets, and receipts inside the database session.

E. Budgeted database session

A budgeted database session is a database session bound to one active task token. The session may issue open-ended SQL, but every attempt is checked against the token, safe view registry, operation policy, and remaining budget.

A connection may bind at most one active task token. Rebinding is denied. When the token expires or is revoked, further `taskbound.run` calls fail.

V. Control Plane: Templates, Applications, Approvals, and Budgets

The control plane is the SaaS-like layer that makes SessionBound realistic for enterprises. This is where SessionBound differs most clearly from database-only security systems.

A. Business users approve tasks, not policies

A data owner should see:

```
Alice requests a June Sales
travel reimbursement anomaly review.
Scope: Sales department, June 2026.
Denied: salary, bank account, phone.
Budget: 20 queries, bounded
entity exposure.
TTL: 30 minutes.
```

The data owner should not see:

```
CREATE POLICY ...
ALTER ROLE ...
GRANT SELECT ON ...
```

The control plane converts the business approval into a signed token that the runtime can enforce.

B. Grants and eligibility

Roles do not grant direct database access. They grant eligibility to request task templates. For example, a finance reviewer may be eligible to request finance-review tasks; a department manager may be eligible to request department-scoped analysis tasks.

This avoids broad database roles such as:

```
finance_user can access all finance tables
```

Instead, the database sees a concrete task token: agent X may analyze June 2026 Sales reimbursements using views A, B, C under budget Y.

C. Budget assignment

Budget assignment is a control-plane decision. A low-risk task might receive a larger aggregate budget; a sensitive task may receive a small detail-row budget and a short TTL. Budgets can be adjusted by role, data classification, task purpose, and approval route.

The database runtime enforces the budget; the control plane decides what budget is appropriate.

VI. SessionBoundDB Runtime

SessionBoundDB is the PostgreSQL-based database runtime for SessionBound.

A. Session binding and credential-token binding

The runtime begins with two objects:

1. a short-lived database credential issued by the credential broker;
2. a signed task token issued by the control plane.

The credential authenticates the runtime connection. The task token authorizes the work.

At `bind_task`, the runtime checks:

- the task token signature is valid;
- the task token is unexpired and not revoked;
- the credential is unexpired and not revoked;
- the token `credential_id` matches the current session credential;
- the token `actor` matches the credential actor;
- the session login role matches the broker-issued database user;
- the token audience is `sessionbounddb`;
- no other active task is already bound to this database backend/session.

A production implementation should make a stolen credential without a matching token insufficient, and a stolen token without the matching credential insufficient. Credential-token binding is intended to reduce cross-task credential misuse.

PostgreSQL nuance: inside `SECURITY DEFINER` functions, `current_user` may refer to the function owner. Therefore, a production implementation should use `session_user` and/or an explicit credential registry mapping rather than relying on `current_user` alone. The measured prototype validates token signatures, token expiration, revoked task state, credential-id-to-token matching, actor matching, audience validation, cross-credential token replay rejection, and same-session rebind rejection in the tested prototype path; Section XIII reports those tests explicitly. Production deployments still require hardened credential brokerage, key management, and audit-retention integration outside this demo.

B. Safe views as the query surface

Agents do not query raw application tables. They query registered safe views. Safe views expose business objects, not internal schemas. This is complementary to database-native mechanisms such as PostgreSQL row-level security policies, which provide row filtering at the database layer [9].

Example safe views:

```
expenses
employees
departments
approval_events
ledger_entries
```

A safe view may:

- join raw tables into business objects;
- remove sensitive fields;
- apply tenant, department, month, or project scope;
- expose workflow hints;
- include stable business identifiers for budget accounting.

Safe views are not a complete inference-control solution. They reduce the discoverable surface and make it governable.

C. Operation policy

`taskbound.run(sql)` is intended for read-only analytical SQL. High-value writes should use controlled commands or stored procedures. The runtime rejects mutations, DDL, unsafe utility commands, raw schema access, denied fields, and blocked schemas.

D. Query decision pipeline

At runtime, every SQL attempt follows the deterministic decision procedure in Algorithm 1.

Algorithm 1. SQL decision pipeline for SessionBoundDB
Input: SQL attempt and bound database session state.
Output: Allowed result with receipt, or denial receipt.

1) Require an active task binding for the session.
2) Validate the signed task token and its TTL.
3) Resolve all referenced relations against registered safe views.
4) Reject denied fields and blocked schemas.
5) Enforce row scope predicates for the bound task.
6) Reject disallowed operation types, including mutations, DDL, and unsafe utility commands.
7) Check remaining query and disclosure budgets.
8) If every check passes, execute the query and emit an allow receipt; otherwise reject the attempt and emit a denial receipt.

The runtime does not ask an LLM whether a query is safe.

VII. Security Invariants

The SessionBoundDB enforcement contract can be described by a state

$$S = \langle T, C, V, B, R \rangle, \tag{1}$$

where T is the signed task token, C is the credential/session binding, V is the safe-view registry and policy version, B is the remaining budget vector, and R is the receipt chain. Each SQL attempt is evaluated by:

$$decide(q, S) \rightarrow allow(result, S') \\ \text{or } deny(reason, receipt, S'). \tag{2}$$

These invariants define the enforcement contract implemented by the prototype and used to structure the adversarial evaluation. They are not a formal proof that all semantic inference is impossible.

- I1. No SQL execution occurs without an active task binding.
- I2. The bound token must match the session credential, actor, audience, expiry, token digest, and revocation state.
- I3. Referenced relations must belong to the approved safe-view registry.
- I4. Direct access to denied fields, raw schemas, catalog escape, mutation, DDL, and blocked payload aggregation is denied.
- I5. Scope predicates or session-bound claims constrain visible rows.
- I6. Budget state is monotonic: an allowed query consumes budget, or the query is denied.
- I7. Every allow/deny decision emits a receipt.
- I8. Safe-view registry, policy version, or definition-hash drift invalidates the token or requires reapproval.

In the prototype, I1–I2 and I8 are checked during `taskbound.bind_task`; I3–I4 are checked by API-layer AST preflight, the experimental `sessionbound_guard` PostgreSQL hook path, and database permissions; I5 is enforced by safe-view predicates; I6 is implemented through task execution state; and I7 is implemented through allow/deny receipts. Production implementations should harden these structural checks into always-on planner or executor integration, but the invariant set is independent of that implementation choice.

A. Security Guarantees for the Prototype SQL Fragment

We state the prototype security contract over a deliberately restricted SQL fragment. Let SELECT-F denote single-statement, read-only SQL consisting of `SELECT`, projection, predicates, joins, `GROUP BY/HAVING`, ordering and limits, non-recursive CTEs, and window functions over registered safe-view names. The fragment excludes stacked statements, DDL, DML, `COPY`, `DO`, `CALL`, `CREATE FUNCTION`, temporary object creation, recursive CTEs, set-operation stacking, table functions, raw-schema or catalog references, and high-risk payload aggregation such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`.

Let $\mathrm{Rels}(q)$ be the set of relation identifiers in q after database name resolution and before safe-view expansion. Let $\mathrm{Views}(T, V)$ be the approved safe views carried by token T and validated against registry V. Let $\mathrm{Cols}_{out}(q)$ be the direct column references that appear in the output expressions of q. A state $S = \langle T, C, V, B, R \rangle$ is well formed when the token is valid and bound to credential C, the safe-view registry and policy hashes match the token, the budget vector B is nonnegative, and the receipt chain R verifies. Under these assumptions, the prototype provides the following direct-reference and accounting guarantees. These are not guarantees against arbitrary semantic inference from allowed answers, and they are not differential privacy.

Proposition 1 (Safe-surface confinement). For any well-formed state S and query $q \in$ SELECT-F, if $decide(q, S) = allow(result, S')$, then $\mathrm{Rels}(q) \subseteq \mathrm{Views}(T, V)$. Consequently, every raw base relation evaluated during query execution is reached only through the definition of a registered safe view approved for the task; the query has no direct reference path to raw tables or catalogs.

Argument. The decision pipeline resolves relation identifiers and checks them against the task's approved safe-view registry before any result is released. Database permissions deny raw-schema access, while the AST preflight and hook path reject resolved relation OIDs outside the approved safe-view set. Therefore any query that names an unregistered relation is denied. Raw base tables may still be read by trusted safe-view definitions, but not by direct agent-supplied SQL.

Proposition 2 (Denied-field non-disclosure for direct references). Let $D(T)$ be the denied-field set in token T, and let $\mathrm{Cols}(v, V)$ be the registered column list for safe view v. If $decide(q, S) = allow(result, S')$, then every direct output column reference $c \in \mathrm{Cols}_{out}(q)$ is in $\bigcup_{v \in \mathrm{Views}(T,V)} \mathrm{Cols}(v, V)$ and $c \notin D(T)$. Thus a column that is absent from the approved safe-view column lists, or that appears in the denied-field set, cannot be directly projected by an allowed query.

Argument. In SELECT-F, output columns must resolve against the relations that survived Proposition 1. A column not exported by an approved safe view cannot be bound by name. A column name or alias in the denied-field set is rejected by the structural policy before execution. The guarantee is syntactic and direct: it does not rule out inferences about denied fields from permitted columns or aggregates.

Proposition 3 (Monotonic budget accounting). Let budgets be ordered componentwise by $\preceq$, and let $\Delta(q, result) \succeq 0$ be the budget charge for a released result. For any allowed query, $decide(q, S) = allow(result, S')$ implies $0 \preceq B' \preceq B$, where B' is the budget component of S'. If a required charge is not covered by the remaining budget, then the query is denied and no result is released.

Argument. The runtime treats budget dimensions as nonnegative counters. Query-count budget is checked before execution, and disclosure budget is charged before release using the accounted result shape and business identifiers. Updates only subtract nonnegative charges from the remaining vector. Therefore the remaining budget cannot increase across an allowed transition; an over-budget

attempt reaches a denial state instead of returning the result.

Proposition 4 (Receipt-chain tamper evidence under trusted DB assumptions). Assume collision-resistant hashing and append-only receipt storage inside the trusted database boundary. For a receipt chain $R = (r_1, \ldots, r_n)$ where each r_i stores the hash of r_{i-1} and a hash of its own canonical contents, removing or modifying any interior receipt r_j, $1 < j < n$, changes r_j's hash or breaks the previous-hash value stored in r_{j+1}. The change is detected by chain verification unless the adversary can violate append-only storage or find a hash collision.

Argument. Each receipt commits to the previous receipt hash and to the current decision, query hash, timestamp, and budget delta. Changing an interior receipt changes its hash; deleting it changes the predecessor expected by the next receipt. Under append-only storage, the adversary cannot rewrite the downstream chain to hide the mismatch. This is a tamper-evidence property for the audit trail, not a Byzantine storage guarantee against malicious DBAs or compromised hosts.

VIII. Anti-Evasion Scope

SessionBound handles direct boundary violations, but it does not claim to eliminate all semantic inference attacks. This distinction is central to the design.

A. Direct boundary violations

SessionBoundDB can directly block:

- access to raw schemas;
- access to sensitive fields that are not exposed through safe views;
- queries outside row scope;
- unsupported operation types;
- repeated querying beyond task budgets;
- use of credentials without a matching task token;
- attempts to continue after token expiry or revocation.

B. Inference and side channels

SessionBound does not claim to solve arbitrary inference from allowed answers. For example, if a safe view exposes enough correlated information, an agent may infer sensitive facts indirectly. Aggregates over small groups may leak more than intended. Timing and resource side channels are also outside the prototype's formal guarantees. Classical database inference-control work studies these risks for statistical databases and repeated query answers [10].

SessionBound's position is therefore:

```
SessionBound reduces and accounts for
the discoverable surface;
it does not eliminate all inference.
```

C. Adversarial SQL patterns

Table I summarizes representative evasion patterns, defenses, and limitations.

This table is not a proof of complete SQL security. It defines the prototype's anti-evasion scope and the path toward production hardening.

Examples of payload aggregation patterns include:

```
SELECT json_agg(e) FROM expenses e;
SELECT array_agg(e.expense_id) FROM expenses e;
SELECT array_agg(row_to_json(e)) FROM expenses e;
SELECT string_agg(employee_name, ',') FROM employees;
```

The JSON / array aggregation case is particularly important. A result that appears to contain one row can still encode a large amount of data in a single JSON, array, XML, or string payload. For this reason, SessionBound budgets must include byte-level and payload-shape limits, not only row counters. A conservative prototype may deny high-risk aggregation functions such as `json_agg`, `array_agg`, `string_agg`, `xmlagg`, and `row_to_json` unless a task template explicitly allows them with a strict byte budget.

IX. Budget Semantics

The phrase disclosure budget can be misleading if interpreted as differential privacy. SessionBound uses disclosure budgets for a different purpose.

Differential privacy protects statistical privacy, often by adding noise and accounting for privacy loss [11].

SessionBound disclosure budgets serve a different purpose. They limit how much task-authorized information an agent may retrieve or inspect during one approved analytical session. They are an authority-accounting mechanism, not a formal differential-privacy guarantee.

A. Budget vector

A production SessionBound deployment should use a budget vector rather than a single unique-row limit.

Example:

```
{
  "max_queries": 20,
  "max_result_rows": 5000,
  "max_result_bytes": 2000000,
  "max_payload_bytes_per_query": 250000,
  "max_json_array_elements": 1000,
  "max_unique_entities": {
    "expense_id": 5000,
    "employee_id": 200,
    "department_id": 20
  },
  "column_weights": {
    "expense_id": 1,
    "department_name": 1,
    "amount": 3,
    "employee_name": 5,
    "approval_reason": 8
  },
  "detail_multiplier": 1.0,
  "aggregate_multiplier": 0.2,
  "small_group_multiplier": 3.0,
  "blocked_or_weighted_functions": {
    "json_agg": "block_or_charge_high",
    "array_agg": "block_or_charge_high",
```

TABLE I
ADVERSARIAL SQL PATTERNS AND SESSIONBOUND DEFENSES.

Pattern	Example	Defense	Limitation
Raw table escape	`SELECT * FROM app_data.expenses`	no raw grants; safe views only	strong in prototype
Sensitive column	`SELECT salary FROM employees`	denied fields; safe view exclusion	direct leakage only
Scope escape	query another month or department	safe-view predicates from task claims	strong if views are correct
Mutation	`DELETE`, `UPDATE`, `INSERT`	operation policy; read-only `run` path	writes require controlled commands
DDL	`DROP`, `ALTER`, `CREATE`	blocked operations	production should use hook-backed validation
Catalog escape	`pg_catalog`, `information_schema`	blocked schemas and no grants	production needs planner/executor hooks
Pagination scraping	repeated `LIMIT/OFFSET`	query budget and disclosure budget	budget model matters
JSON / array payload aggregation	payload aggregation functions	result-byte budget, per-query payload cap, optional aggregate allowlist / AST policy	production should detect high-risk aggregation functions
Aggregate inference	small-group `GROUP BY`	optional minimum group size and aggregate policy	future work
Timing side channel	expensive predicates	statement timeout and resource budget	partial

```
    "string_agg": "block_or_charge_high",
    "xmlagg": "block_or_charge_high"
  }
}
```

B. Budget dimensions

The runtime may account for:

- query count: limits repeated attempts;
- result rows: limits raw output volume;
- result bytes: limits bulk export;
- per-query payload bytes: prevents one-row JSON, array, XML, or string payloads from smuggling a large relation as a single tuple;
- aggregate payload shape: optionally blocks or heavily charges high-risk aggregation functions such as `json_agg`, `array_agg`, `string_agg`, and `xmlagg`;
- unique business entities: tracks exposure of objects such as expenses, employees, tickets, or customers;
- column weights: charges more for more sensitive exposed attributes;
- detail versus aggregate multipliers: charges detail rows more than coarse aggregates;
- small-group penalties: charges more when aggregates describe very small groups.

The prototype's unique-row accounting demonstrates the mechanism. The research model is broader: a budget is a vector of exposure counters tied to task authority.

C. Budget limitations

Budget vectors are not formal mutual-information bounds. They do not prove that an agent cannot infer a sensitive fact from allowed answers. They provide operational containment, auditability, and a practical governance lever for approved enterprise tasks.

X. SCHEMA DRIFT AND SAFE VIEW VERSIONING

Task tokens should not silently survive changes to the data boundary they approve. If a safe view changes after approval, the token may no longer represent the approved task.

A. Binding model

A task token should bind safe views by more than name. For each safe view, the token or registry state should include:

```
safe_view_id
safe_view_registry_version
policy_version
database_object_oid
view_definition_hash
safe_column_list_hash
```

PostgreSQL nuance: ordinary tables have physical storage files, but views do not behave like ordinary stored relations. Binding to `relfilenode` is not appropriate for views. `pg_class.oid` is useful but insufficient: `CREATE OR REPLACE VIEW` may preserve the OID while changing the definition. Therefore, SessionBound should compare view definition hashes and registry versions.

B. Drift behavior

Table II summarizes drift events and required responses.

A task-bound session should fail closed if its safe view registry version or view definition hash no longer matches the approved boundary.

XI. RECEIPTS

SessionBound receipts are structured audit records that bind task identity, query decision, and budget impact. They are not merely logs.

A receipt may contain:

```
{
  "receipt_id": "receipt_789",
  "task_id": "task_456",
  "actor": "agent:travel-expense-analyst",
  "query_hash": "...",
  "views_touched": ["expenses"],
  "decision": "allowed",
  "rows_returned": 120,
  "budget_delta": {
    "queries": 1,
    "result_rows": 120,
    "weighted_disclosure": 240
  },
```

TABLE II
SAFE VIEW DRIFT AND REQUIRED TOKEN INVALIDATION BEHAVIOR.

Change	Risk	Required Response
Add column to raw table	hidden exposure if safe view uses `SELECT *`	safe views must not use `SELECT *`; registry hash unchanged if not exposed
Add column to safe view	new exposure	invalidate existing task tokens
`CREATE OR REPLACE VIEW`	semantics drift	compare view definition hash; invalidate if changed
`DROP/CREATE VIEW`	object replacement	OID mismatch; invalidate
Policy version update	approval drift	invalidate or require explicit compatibility flag
Column type change	interpretation drift	invalidate if exposed column hash changes


```
  "budget_remaining": {
    "queries": 19,
    "weighted_disclosure": 9760
  },
  "timestamp": "2026-06-24T10:05:00Z",
  "previous_receipt_hash": "...",
  "receipt_hash": "..."
}
```


The prototype implements a lightweight hash chain: each receipt stores `previous_receipt_hash` and `receipt_hash`, where the current hash commits to the task id, query hash, decision, budget delta, timestamp, and previous hash. This does not provide a full external proof system, but it makes post-hoc tampering with the receipt sequence detectable within the database audit trail. Future work can explore Merkle receipts, verifiable audit logs, cryptographic accumulators, and external notarization.

XII. Prototype Implementation and Production Hardening

A. Prototype

The current prototype includes:

- FastAPI control plane;
- task template and grant registry;
- short-lived PostgreSQL credential broker;
- signed task token issuance;
- PostgreSQL runtime functions;
- safe views over travel reimbursement data;
- `taskbound.bind_task(payload, signature)`;
- `taskbound.run(sql)`;
- sensitive field denial;
- blocked raw table access;
- query and disclosure budget state;
- receipts for allowed and denied attempts;
- controlled commands for high-value workflow actions;
- an agent-agnostic HTTP evaluation harness.

The prototype demonstrates the SessionBound architecture, not a production-grade SQL safety proof.

B. Implementation honesty

The current hardening prototype uses API-layer AST preflight, conservative database-side checks, a database-resident PostgreSQL hook path, and PostgreSQL permissions. Agents do not receive raw schema grants. Agents call `taskbound.run(sql)`. The runtime role cannot access `app_data` directly. Safe views are the query surface. High-value writes go through controlled commands.

The database hook path is implemented by the `sessionbound_guard` C extension, loaded through `shared_preload_libraries`. The extension registers `post_parse_analyze_hook` and reads trusted SUSET GUCs populated by `taskbound.bind_task`, including the bound task state and the approved safe-view OIDs from the runtime registry. Before dynamic execution, `taskbound.run` calls `public.sessionbound_guard_check(sql)`, which uses `SPI_prepare` to trigger PostgreSQL parse/analyze on the exact SQL text. If the hook denies the query, the runtime emits a denial receipt and raises an error before execution.

The hook rejects non-`SELECT` statements, raw `app_data` relations, catalog access, recursive CTEs, `UNION`/`INTERSECT`/`EXCEPT`, unapproved relation OIDs, non-view relations, unsafe non-catalog functions, sensitive output aliases, and high-risk payload aggregation functions such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`. The API AST validator remains defense in depth and provides portable preflight feedback before the database runtime is invoked.

This should still be read as an experimental hook path, not a production SQL-safety proof. A production implementation should harden the same structural policy into:

- always-on PostgreSQL parser/analyzer hooks;
- planner hooks;
- executor hooks;
- extension-level session state and registry management;
- deny-by-default schema grants;
- blocked unsafe utility commands;
- `statement_timeout` and resource limits;
- cleanup for expired short-lived roles;
- denial logging outside user transactions.

The measured AST prototype extracts referenced relations, columns, function calls, CTEs, recursive CTEs, subqueries, joins, catalog access, operation type, and set operations.

It blocks raw schema references, catalog access, mutations, DDL, `COPY`, `SET`, `SHOW`, `CREATE FUNCTION`, `DO` blocks, recursive CTEs, `UNION/INTERSECT/EXCEPT` stacking, unknown functions, denied-field aliases, and high-risk payload aggregation functions. The API AST validation evaluation passed 17 of 17 query-shape cases, and the PostgreSQL hook evaluation passed 10 of 10 direct database enforcement cases.

C. Query example

Allowed:

```
WITH ranked AS (
  SELECT expense_id, department_name, category, amount,
         row_number() OVER (
           PARTITION BY department_name ORDER BY amount DESC
         ) AS rn
  FROM expenses
  WHERE expense_month = '2026-06'
)
SELECT department_name, expense_id, category, amount
FROM ranked
WHERE rn = 1;
```

Denied:

```
SELECT employee_name, salary FROM employees;
```

Denied:

```
SELECT * FROM app_data.expenses;
```

Denied:

```
DELETE FROM expenses WHERE expense_month = '2026-06';
```

XIII. Evaluation

The prototype evaluation has two goals:

1. validate that approved analytical queries can run over safe views;
2. validate that direct boundary violations are denied and accounted for.

It does not prove production-grade SQL safety or eliminate all inference attacks. The evaluation focuses on functional boundary validation, security-property comparisons against narrower database controls, database-runtime overhead, and concurrency behavior on the default synthetic dataset.

A. Experimental setup

The latest hardening run used Docker Compose with PostgreSQL 16.14, the FastAPI SessionBoundDB prototype, and the `sessionbound_guard` extension loaded through `shared_preload_libraries`. The repository branch was `tdsc-hardening`. The default seed contains 430 expenses, 240 employees, and 124 departments. The raw outputs are stored under `paper/tdsc/raw_results/`.

The AST and adversarial SQL evaluations exercise the HTTP API, including dynamic credential issuance, signed task-token creation, token binding, AST preflight, `taskbound.run`, budget state, and receipts. The hook evaluation bypasses the API query endpoint for agent cases by using generated runtime credentials directly against PostgreSQL, and it uses trusted superuser setup for hook-only structural checks. The overhead breakdown uses direct PostgreSQL connections from the Compose network so that it isolates database runtime overhead from HTTP, model calls, dynamic credential creation, and API-layer AST parsing.

B. Functional validation

We evaluated the SessionBoundDB PostgreSQL prototype using Docker Compose. The public repository contains the evaluation scripts, validation reports, benchmark outputs, and source code needed to reproduce the results. The canonical validation suite passed 24 of 24 scenarios. Table V lists the full canonical suite and includes the corresponding scenario identifiers used by `scripts/sessionbound_agent_eval.py`.

Scope violations over safe views are enforced by filtering rather than explicit denial. A query for another approved-view but out-of-scope month returns zero rows because the safe view predicate binds the session to the task scope. The measured prototype conservatively blocks payload aggregation functions such as `json_agg`, `array_agg`, `string_agg`, and `row_to_json`; ordinary aggregate analysis such as `count`, `sum`, `avg`, `min`, and `max` remains allowed.

C. Baseline security comparison

We compare SessionBound against four narrower database configurations to separate access-surface reduction from task-bound session enforcement:

- **Raw PostgreSQL**: direct access to application tables using a trusted analyst/admin test role.
- **Role-only**: a constrained database role with coarse grants but no task token, budget vector, receipt chain, or task-specific safe view binding.
- **RLS-only**: PostgreSQL row-level security policies over raw tables, without SessionBound task approval, disclosure accounting, or receipt semantics.
- **Safe-view-only**: curated views and denied-field exclusion, without credential-token binding, query budgets, disclosure budgets, or signed task tokens.
- **SessionBound full**: signed task token, short-lived credential binding, safe views, denied fields, operation policy, query budget, disclosure budget, anti-payload aggregation checks, and receipts.

Table VI summarizes the observed security properties. In the updated credential-token test suite, SessionBound denies credential-id mismatch, cross-credential token replay, wrong audience, wrong actor, expired tokens, revoked tasks, and same-session rebind to a second active task. It also rejects tokens whose safe-view registry version, policy version, or view-definition hash no longer matches the runtime registry.

D. Adversarial SQL evaluation

The latest hardening run adds an adversarial SQL suite with 28 cases across direct boundary violations, catalog escape, payload aggregation, obfuscation, CTEs, set-operation stacking, small-group aggregate inference, and search-path/function abuse. The suite passed all expected classifications: 22 cases were blocked, 5 safe-view analytical cases were allowed and accounted for, and 1 small-group aggregate query was classified as a known limitation.

This result should not be read as a proof of complete SQL safety. It does show that the current prototype is no longer limited to literal string checks for the main tested attack families. The remaining small-group aggregate case motivates minimum group-size rules, aggregate sensitivity scoring, and stronger inference-control work.

We also evaluated the database hook path directly. The `sessionbound_guard` suite passed 10 of 10 cases. It confirmed that ordinary agent roles cannot enable trusted guard GUCs, that an approved safe-view `SELECT` succeeds through a direct database credential call, and that `UNION` stacking, catalog access, recursive CTEs, direct runtime-helper abuse, non-`SELECT` SQL, raw schema access, payload aggregation, and unapproved safe-view OIDs are blocked by the database path.

E. Performance overhead and ablation

The latest overhead breakdown measures five query patterns: simple `SELECT`, `JOIN`, `GROUP BY`, CTE, and window function. Each pattern uses 10 warmup iterations and 100 measured iterations per mode. The run compares raw PostgreSQL, a temporary role-only read-only credential, safe-view-only execution, unsupported RLS-only mode, SessionBound without receipts, SessionBound without budget updates, and full SessionBound. Table VII reports p50 latency in milliseconds.

The RLS-only row is marked `N/A` in this run because the current prototype does not define PostgreSQL RLS policies. Earlier experiments separately measured an RLS baseline, but the latest breakdown focuses on currently supported ablations and does not invent an RLS toggle.

The main observation is that most overhead is already present in the safe-view-only mode. Raw and role-only queries are sub-millisecond, while safe-view-only p50 is 13.3–18.1 ms. Full SessionBound p50 is 14.6–18.6 ms. Disabling receipts or budget accounting does not consistently reduce latency, so receipt insertion and budget updates do not dominate this small-dataset benchmark. This is an important interpretation point: for the default seed, SessionBound's measured latency is mainly a safe-view/session-runtime cost rather than a receipt-chain cost.

1) Performance diagnosis: The database-runtime breakdown isolates direct PostgreSQL execution and therefore does not include HTTP latency, model calls, dynamic credential creation, or API-layer AST preflight. End-to-end agent requests do incur the AST preflight before entering PostgreSQL, so the end-to-end path is strictly larger than the database-only numbers in Table VII.

Within the measured database path, the overhead comes from several implementation layers in the reference prototype. First, `taskbound.run` dispatches each query through a PL/pgSQL wrapper rather than through PostgreSQL's native planner and executor path alone. Second, each execution repeatedly resolves session claims and task state instead of caching a bound task descriptor in trusted backend state. Third, safe views add predicate and view-expansion cost even before SessionBound receipt or budget logic runs. Fourth, the wrapper path materializes result rows so that the runtime can account for returned business identifiers. Fifth, receipt insertion and budget accounting add state updates, but the no-receipt and no-budget ablations show that these updates are not the dominant source on the small default dataset.

The scale results in Table VIII show a different regime. At 100k scoped expense rows, safe-view-only execution remains in the tens of milliseconds, while full SessionBound reaches 6.7–7.0 s. This implicates the PL/pgSQL reference runtime's row materialization and per-query accounting path, not safe-view predicates alone. We therefore interpret the prototype as a security reference implementation, not an optimized execution engine. A production implementation should bind the task token once, keep approved safe-view identifiers in trusted backend state, and move structural checks and disclosure accounting into parser/analyzer, planner, or executor hooks.

F. Scale and concurrency

We separately tested credential-token edge cases. The updated prototype denied all tested edge cases: credential A with token B, replaying a token with a second credential, binding a second active task in the same session, wrong token audience, wrong token actor, expired token, and revoked task state. We also tested safe-view drift checks: registry version drift, safe-view policy-version drift, and view-definition hash mismatch were denied and require re-approval before execution.

The default seed contains 430 expenses, 240 employees, and 124 departments. A concurrency smoke test over the API completed 1, 5, and 20 concurrent sessions with zero failed sessions. At 20 concurrent sessions, query p50 was 82.896 ms and query p95 was 110.356 ms. This is an API-stack smoke test, not a throughput-capacity claim.

We also ran a synthetic scale sweep over 1k, 10k, and 100k scoped expense rows. Table VIII reports the p50 latency for two representative query shapes. Raw PostgreSQL stays in the millisecond-to-tens-of-milliseconds range at 100k rows, and safe-view-only execution stays below 33 ms. Full SessionBound reaches 6.7–7.0 s at 100k rows. These results expose the scale-sensitive limitation of the current PL/pgSQL reference runtime and motivate the production hook path discussed above.

TABLE III
ADVERSARIAL SQL EVALUATION SUMMARY FROM THE HARDENING SUITE.

Category	Representative pattern	Result	Notes
Direct boundary violations	denied columns, raw schema, DML, DDL	5 blocked / 5	salary, bank account, `app_data`, `DELETE`, and `DROP` denied
Catalog and metadata escape	`pg_catalog`, `information_schema`, bare `pg_tables`	3 blocked / 3	catalog schemas and known catalog relations denied
Payload aggregation / compression	`json_agg`, `array_agg`, `string_agg`, `row_to_json`	6 blocked / 6	high-risk one-row payload compression denied
Obfuscation and aliases	harmless aliasing, denied-field alias	3 allowed, 1 blocked	allowed queries emitted receipts; `amount AS salary` blocked
Subqueries and CTEs	nonrecursive CTEs and recursive CTE	2 allowed, 1 blocked	ordinary CTEs allowed; recursive/set-operation shape denied
`UNION` and stacking	`UNION`, `UNION ALL`	2 blocked / 2	set-operation stacking denied
Small-group aggregate inference	group by department and employee	1 known limitation	allowed aggregate; no minimum group-size policy yet
Search path / function abuse	`SHOW`, `SET`, `CREATE FUNCTION`, `DO`	4 blocked / 4	unsafe utility/procedural forms denied

TABLE IV
POSTGRESQL HOOK ENFORCEMENT EVALUATION. AGENT CASES BYPASS THE API QUERY ENDPOINT AND CALL POSTGRESQL DIRECTLY AS THE GENERATED RUNTIME CREDENTIAL.

Case group	Enforcement path	Result	Notes
Trusted GUC tamper	Non-superuser direct DB session	1 blocked / 1	Ordinary agent role cannot set SUSET guard GUCs
Safe-view query	Agent credential direct DB call	1 allowed / 1	Approved safe-view relation OIDs accepted
Set operation, catalog, recursive CTE	Agent credential direct DB call	3 blocked / 3	Denied by parser/analyzer hook
Runtime helper abuse	Agent credential direct DB call	1 blocked / 1	Runtime wrapper denies direct helper calls
Hook-only structural checks	Trusted-GUC hook check without API preflight	4 blocked / 4	Non-`SELECT`, raw schema, payload aggregation, and unapproved safe-view OID denied

G. Discussion of remaining gaps

The hardening results improve direct SQL-structure coverage and make the overhead source clearer, but several gaps remain. The PostgreSQL hook path is an experimental parse/analyze guard invoked before dynamic execution; it is not yet production-grade planner or executor enforcement. Small-group aggregate inference remains a known limitation. The budget vector is operational and does not provide formal differential privacy. The PL/pgSQL reference runtime has high scale-sensitive overhead, so a production design should move structural checks and accounting closer to PostgreSQL plan or executor hooks.

H. Threat-to-defense matrix

Table IX summarizes the threat-to-defense mapping.

XIV. Discussion

A. Task approval is not a database policy

SessionBound separates two activities that are often conflated. Business users, managers, data owners, or compliance staff approve tasks. Database runtimes enforce structured boundaries. A finance manager should not need to write SQL predicates, RLS rules, or database policies in order to approve an agent-assisted analysis task. Conversely, the database should not need to understand the business conversation that led to approval.

The control plane is therefore not a convenience layer. It is the place where enterprise intent becomes structured enough for deterministic enforcement: a task template, an application, an approval, a scope, a TTL, a budget vector, and a signed task token. The database runtime consumes this structure and turns it into a bounded session.

B. The runtime should not be an LLM judge

SessionBound deliberately avoids making the database an LLM-based judge of task intent. Natural-language instructions may help a user describe a request, and an agent may use them to plan analysis. However, the runtime does not rely on the agent or any LLM to correctly interpret or obey the task.

This design differs from systems that verify whether a concrete service operation is implied by a natural-language task. SessionBound instead assumes that the approved task has already been reduced to a structured boundary by the control plane. The agent may generate SQL freely inside that boundary, but the database enforces safe views, denied fields, row scope, budgets, and receipts.

TABLE V
CANONICAL FUNCTIONAL VALIDATION SCENARIOS. THE FULL VALIDATION SUITE PASSED 24 OF 24 SCENARIOS.

ID	Paper scenario	Script scenario	SQL Feature / Attack	Expected	Observed
V01	Safe view SELECT	`safe_view_select`	`SELECT`	Allowed	Allowed
V02	Join safe views	`join_safe_views`	`JOIN`	Allowed	Allowed
V03	CTE	`cte`	common table expression	Allowed	Allowed
V04	Department totals	`group_by`	`GROUP BY`	Allowed	Allowed
V05	Ranked expenses	`window_function`	window function	Allowed	Allowed
V06	Scoped drill-down	`scoped_drill_down`	scoped predicate over safe view	Allowed	Allowed
V07	Salary access	`salary_access`	denied column	Denied	Denied
V08	Bank account access	`bank_account_access`	denied column	Denied	Denied
V09	Raw table access	`raw_table_access`	schema escape	Denied	Denied
V10	Mutation SQL	`mutation_sql`	`DELETE`	Denied	Denied
V11	DDL	`ddl`	`DROP`	Denied	Denied
V12	`pg_catalog` access	`pg_catalog_access`	catalog escape	Denied	Denied
V13	`json_agg(e)` payload	`json_agg_payload`	aggregation payload	Denied	Denied
V14	`jsonb_agg(e)` payload	`jsonb_agg_payload`	aggregation payload	Denied	Denied
V15	`array_agg(...)` payload	`array_agg_payload`	aggregation payload	Denied	Denied
V16	`string_agg(...)` payload	`string_agg_payload`	aggregation payload	Denied	Denied
V17	`xmlagg(...)` payload	`xmlagg_payload`	aggregation payload	Denied	Denied
V18	`row_to_json(e)` payload	`row_to_json_payload`	aggregation payload	Denied	Denied
V19	`json_build(...)` payload	`json_build_object_payload`	aggregation payload	Denied	Denied
V20	`jsonb_build(...)` payload	`jsonb_build_object_payload`	aggregation payload	Denied	Denied
V21	Other month	`out_of_scope_month`	scope escape	Transparent scope filtering	Transparent scope filtering
V22	Other department	`out_of_scope_department`	scope escape	Transparent scope filtering	Transparent scope filtering
V23	Query budget overflow	`query_budget_overflow`	excessive queries	Denied	Denied
V24	Disclosure budget overflow	`disclosure_budget_overflow`	too many unique expense rows	Denied	Denied

TABLE VI
SECURITY-PROPERTY COMPARISON ACROSS MEASURED BASELINES. CREDENTIAL-TOKEN BINDING INCLUDES CREDENTIAL-ID MATCHING, ACTOR MATCHING, AUDIENCE VALIDATION, CROSS-CREDENTIAL REPLAY REJECTION, EXPIRATION, REVOCATION, AND SAME-SESSION REBIND REJECTION. SCHEMA DRIFT INVALIDATION CHECKS SAFE-VIEW REGISTRY VERSION, POLICY VERSION, AND VIEW-DEFINITION HASH AT BIND TIME.

Property	Raw	Role-only	Safe-view-only	RLS-only	SessionBound
Blocks writes	No	Yes	Yes	Yes	Yes
Blocks raw table access	No	No	Yes	No	Yes
Blocks denied fields	No	No	Yes	No	Yes
Enforces row scope	No	No	Yes	Yes	Yes
Query budget	No	No	No	No	Yes
Disclosure budget	No	No	No	No	Yes
Payload aggregation blocking	No	No	No	No	Yes
Credential-token binding	No	No	No	No	Yes
Receipts	No	No	No	No	Yes
Schema drift token invalidation	Not tested	Not tested	Not tested	Not tested	Yes

TABLE VII
V3 OVERHEAD BREAKDOWN P50 LATENCY IN MILLISECONDS. THE RUN USED 10 WARMUP AND 100 MEASURED ITERATIONS PER PATTERN AND MODE. RLS-ONLY IS NOT SUPPORTED BY THE CURRENT PROTOTYPE AND IS REPORTED AS N/A.

Pattern	Raw	Role	Safe view	RLS	SB no receipt	SB no budget	SB full
SELECT	0.233	0.229	14.213	N/A	14.899	14.333	14.630
JOIN	0.182	0.171	18.092	N/A	18.497	18.238	18.581
GROUP BY	0.193	0.169	13.296	N/A	14.840	15.074	15.024
CTE	0.176	0.175	13.525	N/A	14.529	15.098	15.425
Window	0.351	0.347	13.828	N/A	15.089	14.852	15.854

C. Filtering, denial, and SQL semantics

Not every boundary violation needs to look like an error. For scoped safe views, transparent filtering is often the desired behavior. A query for another month or department may return zero rows because the safe-view predicate binds the session to the approved task scope. This preserves normal SQL semantics while preventing out-of-scope disclosure.

By contrast, attempts to leave the approved query surface should be explicitly denied. Raw table access, denied fields, mutation statements, DDL, unsafe catalog access, and blocked payload aggregation functions are treated as boundary violations. This distinction lets SessionBound support ordinary analytical SQL while still refusing attempts to escape the task session.

D. Budgets as authority accounting

SessionBound disclosure budgets are not differential privacy guarantees. They do not prove that no inference is possible,

TABLE VIII
SYNTHETIC SCALE SWEEP P50 LATENCY IN MILLISECONDS. SAFE DENOTES SAFE-VIEW-ONLY EXECUTION; SB DENOTES FULL SESSIONBOUND.

Rows	Query	Raw	Safe	RLS	SB
1k	Agg.	0.41	0.60	0.39	84.41
1k	Top-k	0.40	0.53	0.43	82.44
10k	Agg.	2.70	3.76	2.38	688.66
10k	Top-k	2.54	3.32	2.39	661.62
100k	Agg.	14.77	29.96	15.08	6742.55
100k	Top-k	15.62	32.39	14.61	6965.09

TABLE IX
THREAT-TO-DEFENSE MAPPING FOR THE SESSIONBOUND PROTOTYPE.

Threat	SessionBound Defense
Agent ignores task	database enforces signed task token
Prompt injection	safe views, denied fields, budgets
Raw table access	no raw schema grants
Sensitive field access	denied columns and safe view exclusion
Scope escape	row scope in safe views
Unauthorized writes	read SQL blocked; writes via controlled commands
Query flood	query budget
Cumulative detail exposure	disclosure budget vector
Credential leakage	short-lived DB credential plus required task-token binding; credential-id, actor, audience, replay, and rebind checks are enforced in the tested path
Audit gap	task receipts
Schema drift	safe view registry version, policy version, and definition hash checks invalidate stale task tokens

and they do not add noise to query results. Instead, they are an operational accounting mechanism for delegated task authority.

This distinction matters in enterprise settings. A business approval often means: this agent may inspect enough data to perform this task, but not unlimited data. Budget vectors make this approval measurable through query count, result rows, result bytes, unique business entities, column weights, and aggregate/detail multipliers. Receipts then record how the approved authority was spent.

E. Composition with existing authorization systems

SessionBound is intended to compose with, not replace, existing authorization infrastructure. Agent identity systems and authenticated delegation can establish who the agent is and who delegated authority. PAuth-style operation authorization can protect specific service or tool operations. Database security systems such as RLS or Oracle-style policy engines can provide low-level enforcement mechanisms. Data product catalogs can help users discover governed data products.

SessionBound occupies the layer between enterprise task approval and database execution. It turns an approved analytical task into a short-lived, budgeted, auditable database session. This contract gives the agent room to explore while giving the enterprise a deterministic boundary to enforce and audit.

XV. RELATED WORK

A. ABAC, XACML, capabilities, and purpose-based access control

Attribute-based access control and XACML provide mature ways to express authorization policies over subject, object, action, and environmental attributes [12], [13]. Capability systems represent authority as an unforgeable object or reference rather than as ambient identity [14], [15]. Purpose-aware database privacy systems, including Hippocratic databases and purpose-based relational access control, connect data use to stated purpose and policy metadata [16], [17].

SessionBound is related to all three lines, but it uses a different operational object. The unit of approval is an enterprise task, not a standing role, a general policy rule, or a data subject's privacy preference. The task token behaves partly like a capability, but it is bound to a short-lived database credential, safe-view registry, budget vector, and receipt chain.

B. OAuth, agent identity, and authenticated delegation

OAuth 2.0 and related standards support delegated API access [1]. Agent identity systems make agents visible and manageable as first-class non-human identities. Authenticated delegation frameworks make it possible to verify that an agent is acting on behalf of a user and that a credential carries scope [2].

SessionBound can build on these systems. Authenticated Delegation establishes who may act for whom; SessionBound turns approved enterprise tasks into enforceable database sessions.

C. Prompt injection and agent security

Prompt injection work shows that LLM-integrated applications blur the line between data and instructions. Indirect prompt injection can make retrieved content influence tool calls and data disclosure, and prompt injection has become a recognized top risk for LLM applications [18], [19], [20]. Recent work also explores information-flow control for AI agents as a way to give stronger guarantees about how labeled information affects actions [21], [22].

SessionBound addresses this risk by removing the LLM from the final database authorization decision. Prompt injection may cause an agent to try unsafe SQL, but the database session is still bound to safe views, denied fields, operation limits, budgets, and receipts.

D. PAuth and task-scoped service operations

PAuth provides precise task-scoped authorization for service or tool operations [3]. It derives NL slices from a task and checks that concrete operations and operands match task-implied computations.

SessionBound is complementary. It does not attempt to verify that every SQL query is semantically implied by a natural-language request. Instead, it enforces the approved task boundary over open-ended SQL using structured tokens, safe views, budgets, and receipts. In short, PAuth checks whether an operation is implied by the task description; SessionBound checks whether an agent's SQL exploration stays within a pre-approved, structured task boundary.

E. Data Product MCP and enterprise data product governance

Data Product MCP integrates data-product marketplaces and MCP so agents can discover, request access to, and query enterprise data products [4]. The Model Context Protocol standardizes tool connectivity for agent systems [5]. Data Product MCP is the closest related work to SessionBound's enterprise governance motivation.

SessionBound differs by making the approved task session, not the data product, the primary runtime object. A SessionBound session may span multiple safe views and account for cumulative exposure across queries. Its core abstraction is the budgeted database session created from task approval. In short, Data Product MCP governs access to data products; SessionBound governs the execution session of an exploratory agent.

F. Database policy mechanisms, query structure, and auditing

Database-native mechanisms such as PostgreSQL row-level security and Oracle Virtual Private Database attach policy enforcement to data access [9], [23]. SQL injection defenses such as SQLrand and SQLCHECK show the value of query-structure analysis and parser-mediated validation for command integrity [24], [25]. Query auditing studies when sequences of database answers disclose protected information [26], [27]. DLP and data leakage prevention systems focus on detecting or preventing unauthorized sensitive-data movement across endpoints, networks, and storage [28], [29].

SessionBound composes these ideas for agent-generated SQL. It uses database-side safe views and grants for the data-plane boundary, API-layer AST preflight plus an experimental PostgreSQL hook path for query-shape checks in the hardening prototype, and receipts plus budgets for task-session accountability. Unlike generic DLP, SessionBound accounts for authority spent inside an approved database task session rather than inspecting all enterprise data flows.

G. Oracle DDS and database-enforced access control

Oracle Deep Data Security and related systems show that database-enforced access control for agentic AI, AI-assisted applications, direct exploratory access, and analytics is an important industry direction [6], [7]. Such systems are strong data-plane enforcement mechanisms.

SessionBound does not claim that this broad direction is new. It adds an enterprise task control plane and task-bound session model around database enforcement: templates, applications, approvals, signed task tokens, budgets, receipts, and safe-view versioning. In short, Oracle DDS enforces identity- and context-aware database policies; SessionBound provides the task control plane that turns business approval into a bounded, budgeted database session without requiring business users to write SQL predicates.

H. Zanzibar and relationship-based authorization

Zanzibar provides a consistent, global relationship-based authorization system [8]. It performs relationship-based authorization checks at global scale.

SessionBound models budgeted database sessions for agent-generated analysis: once an enterprise task is approved, how should an agent's database session be bounded, budgeted, and audited while it generates SQL?

I. Differential privacy, inference control, query auditing, and DLP

Differential privacy and database inference control address leakage from query answers, often in statistical settings [11], [10]. Query auditing systems track when prior answers would reveal protected facts, while DLP systems focus on sensitive-data movement across enterprise channels.

SessionBound is related but does not claim formal DP guarantees. Its disclosure budget is operational accounting for delegated task authority. It limits cumulative exposure during a task session and produces receipts for governance and audit.

XVI. Future Work

SessionBound opens several research and engineering directions beyond the first prototype.

First, cross-database and lakehouse federation would extend task-bound sessions beyond a single PostgreSQL runtime. A production deployment may need to bind one approved task to Snowflake, BigQuery, PostgreSQL, vector stores, and lakehouse tables while preserving one budget, one approval record, and one receipt chain.

Second, production-grade SQL enforcement should harden the current database hook prototype into planner- or executor-level enforcement inside PostgreSQL. The hardening prototype already includes database-resident structural validation through a `post_parse_analyze_hook`; future work should make this path always on for task sessions,

add deeper aggregate-function and catalog-access policies, and handle generated SQL equivalence more robustly.

A concrete PostgreSQL production path is to bind the task token once at session start, keep approved safe-view OIDs in trusted backend state, inspect the resolved query tree after name resolution, and then attach task identifiers to the plan or executor state. Planner or executor hooks can account for returned business-object identifiers and emit receipts without routing every query through a PL/pgSQL wrapper. This design is the natural next step suggested by the 100k-row scale sweep.

Third, stronger disclosure accounting is needed. The current budget-vector model is operational rather than formal. Future work should explore sensitivity-aware budgets, aggregate leakage models, minimum group-size policies, reconstruction-risk detection, and possible integration with differential-privacy mechanisms where statistical guarantees are required.

Fourth, verifiable receipts can strengthen the audit layer. The prototype can use structured or hash-chained receipts; future versions could use Merkle trees, append-only transparency logs, cryptographic accumulators, or external notarization to make query receipts tamper-evident across organizational boundaries.

Fifth, safe-view authoring and verification should become a first-class developer workflow. Data platform teams need tools to lint safe views, detect accidental `SELECT *`, compare view-definition hashes, test adversarial SQL patterns, and verify that view changes invalidate or reissue affected task tokens.

Finally, SessionBound raises a broader question: how should enterprise software represent work units for AI agents? Task templates, approvals, budgets, receipts, and handoffs may become common infrastructure for agentic enterprise systems, not just database analysis.

XVII. Limitations

SessionBound is a research prototype and reference architecture.

Current limitations include:

- SQL validation now includes AST-level API preflight and an experimental PostgreSQL parse/analyze hook path, but it is not production-grade planner/executor-level enforcement;
- the prototype targets a single PostgreSQL database;
- HMAC/demo signing should be replaced with KMS/JWKS-backed signing;
- disclosure budgets are operational, not formal DP;
- semantic inference is reduced and accounted for, not eliminated;
- out-of-scope predicates over safe views return no rows through transparent scope filtering rather than producing an explicit denial;
- payload aggregation functions are conservatively blocked rather than governed by a per-template allowlist in the measured prototype;
- safe views require careful review and versioning;
- view drift invalidation is implemented through registry-version, policy-version, and view-definition hash checks, but production deployments still need operational migration and re-approval workflows;
- strict credential-id binding, actor matching, audience validation, token replay rejection across a second credential, and same-session rebind rejection are implemented in the measured prototype, but require production-grade credential brokerage, key management, and audit retention outside this demo;
- current performance results include microbenchmarks, overhead ablations, and a prior 1k/10k/100k synthetic scale sweep, but should not be generalized to production workloads or an optimized planner/executor-hook implementation;
- denial receipts may require out-of-transaction logging in production;
- cross-database federation and lakehouse enforcement are future work;
- malicious DBAs and compromised hosts are out of scope.

XVIII. Conclusion

SessionBound turns enterprise task approval into budgeted database sessions for AI agents. It addresses a gap between business governance and database enforcement: business users approve tasks, data platforms register safe views, agents generate SQL, and databases enforce the approved boundary.

SessionBound is not a replacement for OAuth, authenticated delegation, PAuth, Data Product MCP, Oracle DDS, Zanzibar, or differential privacy. It is a complementary architecture for enterprise-internal analytical work: temporary tasks that are too flexible for fixed SaaS screens and too sensitive for raw database access.

SessionBoundDB, our PostgreSQL runtime, demonstrates how signed task tokens, safe views, denied fields, row scope, query budgets, disclosure budgets, and receipts can be bound to one database session. The result is a practical contract between enterprise task approval and agentic database execution:

```
The agent can try. The database decides.
```

The performance results sharpen the implementation lesson. On the default dataset, full SessionBound adds little over safe-view-only execution, so the small-scale overhead is primarily safe-view and session-runtime overhead rather than receipt-chain insertion or budget updates. At 100k scoped rows, however, the PL/pgSQL reference runtime becomes multi-second while raw PostgreSQL and safe-view-only execution remain much faster. This does not change the core task-bound session model, but it does define the

production engineering path: move from the wrapper-based reference runtime toward parser/analyzer, planner, and executor integration inside PostgreSQL.

References

[1] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6749

[2] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated Delegation and Authorized AI Agents," 2025. [Online]. Available: https://arxiv.org/abs/2501.09674

[3] R. K. Sharma, L. Jiang, Z. Lin, and S. Chen, "PAuth - Precise Task-Scoped Authorization For Agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.17170

[4] M. Tonnarelli, F. Scaramuzza, S. Harrer, and L. W. Dietz, "Data Product MCP: Chat with your Enterprise Data," 2026. [Online]. Available: https://arxiv.org/abs/2601.08687

[5] Model Context Protocol, *Model Context Protocol Specification*, 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18

[6] Oracle, *What Is Oracle Deep Data Security*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/26/ddscg/what-is-oracle-deep-data-security.html

[7] ——, *Oracle Deep Data Security Technical Brief*, Oracle, 2026. [Online]. Available: https://www.oracle.com/a/ocom/docs/security/deep-data-security-technical-brief.pdf

[8] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, and M. Wang, "Zanzibar: Google's Consistent, Global Authorization System," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, 2019, pp. 33–46. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/pang

[9] PostgreSQL Global Development Group, *Row Security Policies*, PostgreSQL, 2026. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[10] N. R. Adam and J. C. Wortmann, "Security-Control Methods for Statistical Databases: A Comparative Study," *ACM Computing Surveys*, vol. 21, no. 4, pp. 515–556, 1989. [Online]. Available: https://doi.org/10.1145/76894.76895

[11] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9. [Online]. Available: https://doi.org/10.1561/0400000042

[12] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," National Institute of Standards and Technology, Special Publication 800-162, 2019. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-162

[13] OASIS XACML Technical Committee, *eXtensible Access Control Markup Language (XACML) Version 3.0*, OASIS, 2013. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[14] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966. [Online]. Available: https://doi.org/10.1145/365230.365252

[15] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," Johns Hopkins University Systems Research Laboratory, Tech. Rep. SRL2003-02, 2003. [Online]. Available: https://classpages.cselabs.umn.edu/Fall-2021/csci5271/papers/SRL2003-02.pdf

[16] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Hippocratic Databases," in *Proceedings of the 28th International Conference on Very Large Data Bases*, 2002, pp. 143–154. [Online]. Available: https://www.vldb.org/conf/2002/S05P02.pdf

[17] J.-W. Byun and N. Li, "Purpose Based Access Control for Privacy Protection in Relational Database Systems," *The VLDB Journal*, vol. 17, no. 4, pp. 603–619, 2008. [Online]. Available: https://doi.org/10.1007/s00778-006-0023-0

[18] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023. [Online]. Available: https://doi.org/10.1145/3605764.3623985

[19] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt Injection Attack against LLM-integrated Applications," 2023. [Online]. Available: https://arxiv.org/abs/2306.05499

[20] OWASP GenAI Security Project, *LLM01:2025 Prompt Injection*, OWASP, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[21] A. C. Myers and B. Liskov, "A Decentralized Model for Information Flow Control," in *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, 1997, pp. 129–142. [Online]. Available: https://www.cs.cornell.edu/andru/papers/iflow-sosp97/paper.html

[22] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI Agents with Information-Flow Control," 2025. [Online]. Available: https://arxiv.org/abs/2505.23643

[23] Oracle, *Using Oracle Virtual Private Database to Control Data Access*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/using-oracle-vpd-to-control-data-access.html

[24] S. W. Boyd and A. D. Keromytis, "SQLrand: Preventing SQL Injection Attacks," in *Applied Cryptography and Network Security*. Springer, 2004, pp. 292–302. [Online]. Available: https://doi.org/10.1007/978-3-540-24852-1_21

[25] Z. Su and G. Wassermann, "The Essence of Command Injection Attacks in Web Applications," in *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2006, pp. 372–382. [Online]. Available: https://doi.org/10.1145/1111037.1111070

[26] J. Kleinberg, C. Papadimitriou, and P. Raghavan, "Auditing Boolean Attributes," in *Proceedings of the Nineteenth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2000. [Online]. Available: https://doi.org/10.1145/335168.335210

[27] S. U. Nabar, B. Marthi, K. Kenthapadi, N. Mishra, and R. Motwani, "Towards Robustness in Query Auditing," in *Proceedings of the 32nd International Conference on Very Large Data Bases*, 2006, pp. 151–162. [Online]. Available: https://www.vldb.org/conf/2006/p151-nabar.pdf

[28] S. Alneyadi, E. Sithirasenan, and V. Muthukkumarasamy, "A Survey on Data Leakage Prevention Systems," *Journal of Network and Computer Applications*, vol. 62, pp. 137–152, 2016. [Online]. Available: https://doi.org/10.1016/j.jnca.2016.01.008

[29] National Institute of Standards and Technology, *Data Loss Prevention*, NIST Computer Security Resource Center Glossary, 2026. [Online]. Available: https://csrc.nist.gov/glossary/term/data_loss_prevention